[笔记]28个Jupyter-Notebook技巧 (//zodiac911.github.io/blog/jupyter-tricks.html)

“人生苦短，我用Python”

Python语言被很多人推崇的原因之一是Python社区贡献的各种各样的库和工具。无论你想做什么，基本上都能找到对应的库，虽然不一定完成度有多好。Jupyter Notebook是其中提及率很高的一款强大工具，可以将代码运行的阶段性结果如图片表格等像笔记一样保存起来最终形成可阅读的笔记。

Jupyter Notebook原名Ipython Notebook，Jupyter这个名字据说取自其支持的三种语言 **JU**lia, **PY**Thon,和 **R**，也可能是来自行星Jupiter即木星。

Jupyter Notebook以服务器的形式运行，以浏览器窗口作为前端与用户交互相对h传统IDE来说更加灵活，社区也贡献了一些插件，使其具有不错的扩展性。

阅读博文28 Jupyter Notebook tips, tricks, and shortcuts (https://www.dataquest.io/blog/jupyter-notebook-tips-tricks-shortcuts/)的时候一遍看一遍敲，就记下这许多。其实基本上就是原文的简略版，加了一点自己的理解。

这样的总结博文里的内容，里面的Feature肯定不是全部都用的到，就算只用其中几个，能提升效率也是极好的。希望每次重读的时候，能多发掘一两个好用的功能。

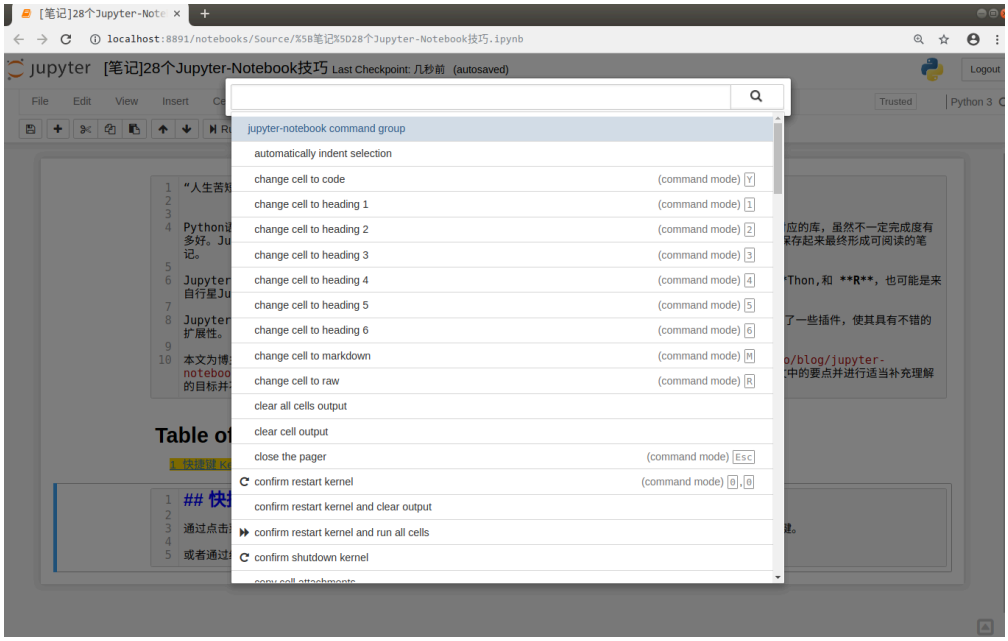
Table of Contents

- 1 快捷键 Keyboard Shortcuts
- 2 显示所有单行变量
- 3 快速文档Easy links to documentation[和`Shift+Tab`功能相同]
- 4 Notebook页内绘图Plotting in notebooks
- 5 魔法命令IPython Magic Commands
- 6 `%env` 设置环境变量
- 7 `%run` 运行外部Python代码
- 8 `%load` 从外部脚本插入代码
- 9 `%store` 在笔记间传递变量
- 10 `%who` 列出所有全局变量
- 11 魔法命令-代码计时
- 12 `%%writefile` / `%pycat` 导出/导入外部脚本
- 13 %pdb调试
- 14 `%prun` 分析代码运行时间
- 15 绘图高分辨率显示
- 16 不显示结果Suppress the output of a final function.
- 17 执行Shell命令
- 18 使用Latex公式
- 19 Notebook内用不同kernel运行代码Run code from a different kernel in a notebook
- 20 安装其他kernel
- 21 同时运行R和Python代码
- 22 用其他语言写部分函数Writing functions in other languages
- 23 多光标
- 24 Jupyter 插件
- 25 利用Jupyter Notebook制作Presentation
- 26 Jupyter输出系统The Jupyter output system
- 27 大数据处理'Big data' analysis
- 28 共享笔记 Sharing notebooks
- 29 补充
    - 29.1 自动补全

快捷键 Keyboard Shortcuts

通过点击菜单中的`Help > Keyboard Shortcuts`或在命名模式下按`H`键都可以查看Jupyter Notebook的快捷键。

或者通过组合快捷键`Ctrl + Shift + P`调出command palette来搜索相应的功能，如图所示



一些常用快捷键：

- `Esc` 进入命令模式
- `Enter` 从命令模式进入编辑模式
- `Esc+`
    - `A` 在当前cell上方插入cell `B` 在当前cell下方插入cell
    - `M` 改变当前cell为Markdown模式 `Y` 改变当前cell为code模式
    - `D+D` 删除当前cell
- `Shift + Tab` 产看当前函数/变量/类的Docstring，可以多次点按调出不同详细程度的Docstring
- `Ctrl + Shift + -` 从光标处拆分当前cell
- `Esc + F` 查找 替换
- `Esc + O` 折叠输出
- 选中多个cell
    - `Shift + J` 或 `Shift + Down` 朝下选择cell `Shift + K` or `Shift + Up` 朝上选择cell 一旦选中多个cell，就可以像对单单一cell一样进行删除/复制/剪切/粘贴/运行了
    - `Shift + M` 合并多个cell

显示所有单行变量

Jupyter Notebook运行后会自动打印cell**最后一行**的变量

通过修改`ast_note_interactivity`变量，Jupyter Notebook可以显示**所有单行变量**

In [1]:

```
import numpy as np
import pandas as pd

ndarray = np.random.random((20, 10))
df = pd.DataFrame(ndarray)

df.head()
df.tail()
```

Out[1]:

	0	1	2	3	4	5	6	7	8	
15	0.446562	0.463690	0.459070	0.171993	0.215920	0.965723	0.782450	0.747307	0.192942	0.21
16	0.124930	0.906744	0.182524	0.243859	0.876654	0.122710	0.561425	0.483087	0.946543	0.49
17	0.729198	0.949683	0.257005	0.649541	0.634265	0.313513	0.632183	0.649272	0.973694	0.91
18	0.438571	0.668593	0.267502	0.780528	0.302337	0.170136	0.559522	0.427518	0.773165	0.42
19	0.422269	0.672814	0.971147	0.051259	0.730094	0.309480	0.781051	0.516392	0.864189	0.93

In [2]:

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

In [3]:

```
df.head()

df.tail()
```

Out[3]:

	0	1	2	3	4	5	6	7	8	
0	0.356586	0.239147	0.096859	0.415080	0.061836	0.230235	0.049476	0.549409	0.384686	0.228
1	0.637844	0.536629	0.423872	0.430975	0.663813	0.580606	0.619456	0.744648	0.314659	0.483
2	0.197630	0.074834	0.184019	0.830379	0.981461	0.524204	0.132249	0.953380	0.687411	0.528
3	0.111197	0.280787	0.154506	0.677373	0.347157	0.898756	0.143879	0.985793	0.790557	0.469
4	0.933522	0.893156	0.476182	0.929917	0.946701	0.649389	0.137161	0.329882	0.975306	0.797

Out[3]:

	0	1	2	3	4	5	6	7	8	
15	0.446562	0.463690	0.459070	0.171993	0.215920	0.965723	0.782450	0.747307	0.192942	0.21
16	0.124930	0.906744	0.182524	0.243859	0.876654	0.122710	0.561425	0.483087	0.946543	0.49
17	0.729198	0.949683	0.257005	0.649541	0.634265	0.313513	0.632183	0.649272	0.973694	0.91
18	0.438571	0.668593	0.267502	0.780528	0.302337	0.170136	0.559522	0.427518	0.773165	0.42
19	0.422269	0.672814	0.971147	0.051259	0.730094	0.309480	0.781051	0.516392	0.864189	0.93

如果想全局设置，执行Shell命令

```
ipython profile create
```

在`~/.ipython/profile_default/`生成ipython_config.py和ipython_kernel_config.py文件，修改ipython_config.py添加如下行

```
c = get_config()

# Run all nodes interactively
c.InteractiveShell.ast_node_interactivity = "all"
```

快速文档Easy links to documentation[和`Shift+Tab`功能相同]

`Help`菜单中有各种在线文档链接

如果像查找具体函数/方法的文档，在其前面添加`?`并运行

In [4]:
```
?str.replace()
```

```
Signature: str.replace(self, old, new, count=-1, /)
Docstring:
Return a copy with all occurrences of substring old replaced by new.

  count
    Maximum number of occurrences to replace.
    -1 (the default value) means replace all occurrences.

If the optional argument count is given, only the first count occurrences are
replaced.
Type:      method_descriptor
```

会弹出Docstring

这一方法和前文提到的`Shift+Tab`功能相同

Notebook页内绘图Plotting in notebooks

- matplotlib `%matplotlib inline`魔法命令使得matplotlib绘图可以在笔记内显示
- `%matplotlib notebook` 为交互式绘图，会有些慢
- Seaborn 只需`import seaborn`就可以让matplot的图好看一些
- mpld3
- plot.ly
- Altair

魔法命令IPython Magic Commands

上文所述`%matplotlib inline`就是一个魔法命令

In [5]:
```
# 列出所有魔法命令
%lsmagic
```

Out[5]:
```
Available line magics:
%alias  %alias_magic  %autocall  %automagic  %autosave  %bookmark  %cat  %cd  %clea
r  %colors  %config  %connect_info  %cp  %debug  %dhist  %dirs  %doctest_mode  %ed
%edit  %env  %gui  %hist  %history  %killbgscripts  %ldir  %less  %lf  %lk  %ll  %l
oad  %load_ext  %loadpy  %logoff  %logon  %logstart  %logstate  %logstop  %ls  %lsm
agic  %lx  %macro  %magic  %man  %matplotlib  %mkdir  %more  %mv  %notebook  %page
%pastebin  %pdb  %pdef  %pdoc  %pfile  %pinfo  %pinfo2  %popd  %pprint  %precision
%profile  %prun  %psearch  %psource  %pushd  %pwd  %pycat  %pylab  %qtconsole  %qui
ckref  %recall  %rehashx  %reload_ext  %rep  %rerun  %reset  %reset_selective  %rm
%rmdir  %run  %save  %sc  %set_env  %store  %sx  %system  %tb  %time  %timeit  %una
lias  %unload_ext  %who  %who_ls  %whos  %xdel  %xmode

Available cell magics:
%%!  %%HTML  %%SVG  %%bash  %%capture  %%debug  %%file  %%html  %%javascript  %%js
%%latex  %%markdown  %%perl  %%prun  %%pypy  %%python  %%python2  %%python3  %%ruby
%%script  %%sh  %%svg  %%sx  %%system  %%time  %%timeit  %%writefile

Automagic is ON, % prefix IS NOT needed for line magics.
```

以下为一些非常有用的魔法命令

`%env` 设置环境变量

无需重启Notebook即可修改环境变量

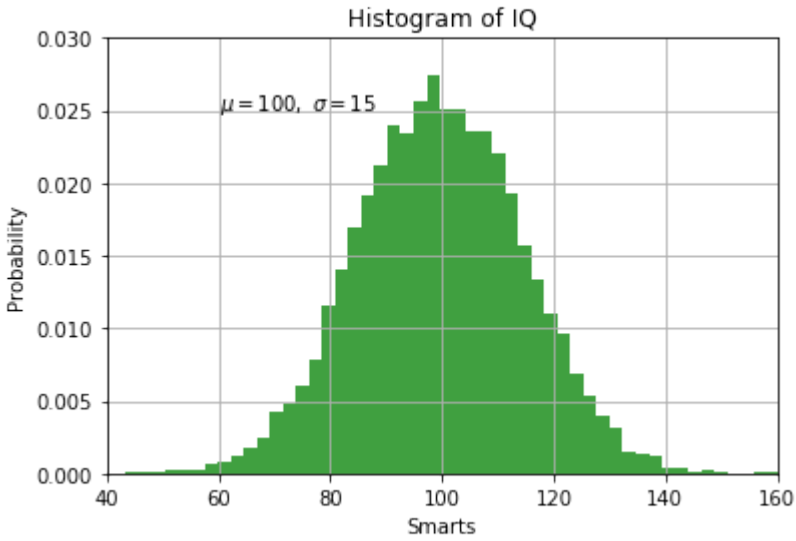
In [6]:
```
%env OMP_NUM_THREADS=4
```

```
env: OMP_NUM_THREADS=4
```

`%run` 运行外部Python代码

注意，这个导入模块不同，仅仅是运行代码

In [7]:
```
% matplotlib inline
%run ./two-histograms.py
```



`%load` 从外部脚本插入代码

这一命令执行后会在当前cell的自动插入外部脚本的代码

执行前

```
%load ./two-histograms.py
```

执行后

```
# %load ./two-histograms.py
import numpy as np
import matplotlib.pyplot as plt

# Fixing random state for reproducibility
np.random.seed(19680801)

mu, sigma = 100, 15
x = mu + sigma * np.random.randn(10000)

# the histogram of the data
n, bins, patches = plt.hist(x, 50, density=True, facecolor='g', alpha=0.75)


plt.xlabel('Smarts')
plt.ylabel('Probability')
plt.title('Histogram of IQ')
plt.text(60, .025, r'$\mu=100,\ \sigma=15$')
plt.axis([40, 160, 0, 0.03])
plt.grid(True)
plt.show()
```

`%store` 在笔记间传递变量

在原Notebook中

In [8]:
```
data = 'this is the string I want to pass to different notebook'
%store data
del data # 删除变量
```

```
Stored 'data' (str)
```

在新的Notebook中

```
%store -r data # 重新载入变量
print(data)
```

this is the string I want to pass to different notebook

`%who` 列出所有全局变量

可以指定数据类型

In [9]:
```
one = "for the money"
two = "for the show"
three = "to get ready now go cat go"

%who str
%who int
```

```
one      three   two
mu       sigma
```

魔法命令-代码计时

`%%time` 与 `%%timeit`

`%%time` 单次执行

`%%timeit` 调用Python timeit 模块，默认执行100000次

这两个魔法命令都可以使用 `%` 或 `%%` 区别在于单行还是整个cell

In [10]:
```
%%time

import numpy as np

ndarray = np.random.random(100)
```

```
CPU times: user 46 µs, sys: 62 µs, total: 108 µs
Wall time: 39.6 µs
```

In [11]:
```
%timeit ndarray = np.random.random(100)
```

```
1.5 µs ± 40 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

In [12]:
```
%%timeit

ndarray = np.random.random(100)
ndarray1 = np.random.random(100)
```

```
2.92 µs ± 83.6 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
```

`%%writefile` / `%pycat` 导出/导入外部脚本

`%%writefile` 将当前cell的内容导出到外部文件中，`%pycat` 正好相反(在弹出窗口显示)

In [13]:
```
%%writefile pythoncode.py

import numpy
def append_if_not_exists(arr, x):
    if x not in arr:
        arr.append(x)

def some_useless_slow_function():
    arr = list()
    for i in range(10000):
        x = numpy.random.randint(0, 10000)
        append_if_not_exists(arr, x)
```

```
Overwriting pythoncode.py
```

In [14]:
```
%pycat pythoncode.py
```

%pdb调试

调用pdb 模块的魔法命令

个人意见：不如 `set_trace()` 好用

```
%pdb

import numpy

def pick_and_take():

    picked = numpy.random.randint(0, 1000)
    raise NotImplementedError()

pick_and_take()
```

`%prun` 分析代码运行时间

在弹出窗口显示每个函数的代码运行时间

In [15]:
```
def useless(n):
    return [x for x in range(n) if x % 2 == 0]
```

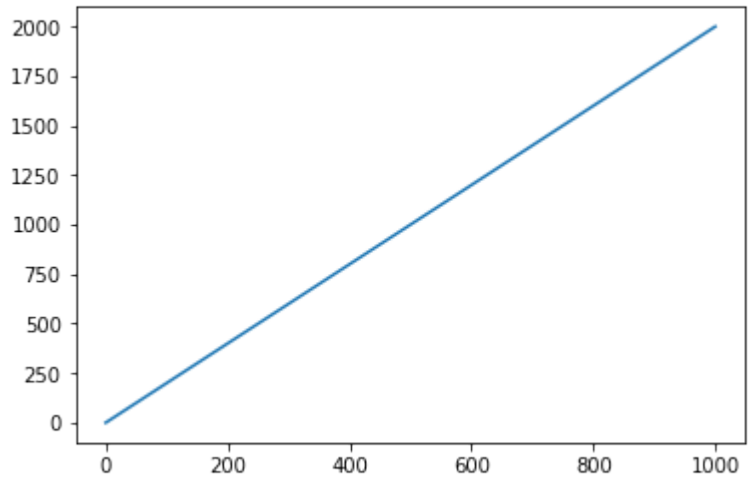
In [16]:
```
%%prun

useless(100)
```

```
         5 function calls in 0.000 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
        1    0.000    0.000    0.000    0.000 <ipython-input-1-5b23640614a7>:2(<listcomp>)
        1    0.000    0.000    0.000    0.000 <ipython-input-1-5b23640614a7>:1(useless)
        1    0.000    0.000    0.000    0.000 <string>:3(<module>)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

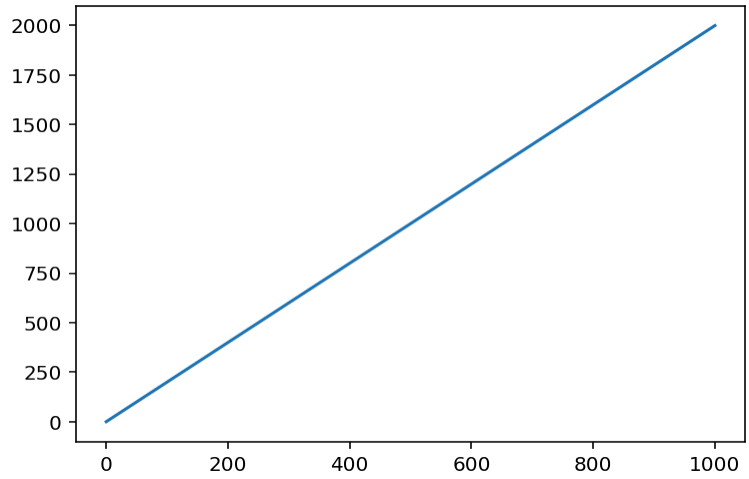
绘图高分辨率显示

`%config InlineBackend.figure_format = 'retina'`

In [17]:
```
x = np.linspace(0, 1000)
y = 2*x
plt.plot(x,y)
plt.show();
```



In [18]:
```
%config InlineBackend.figure_format = 'retina'
plt.plot(x,y)
plt.show();
```



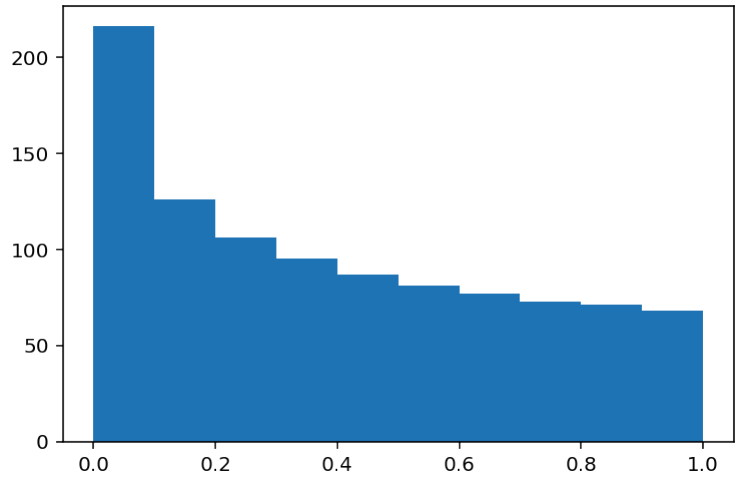
不显示结果Suppress the output of a final function.

加分号

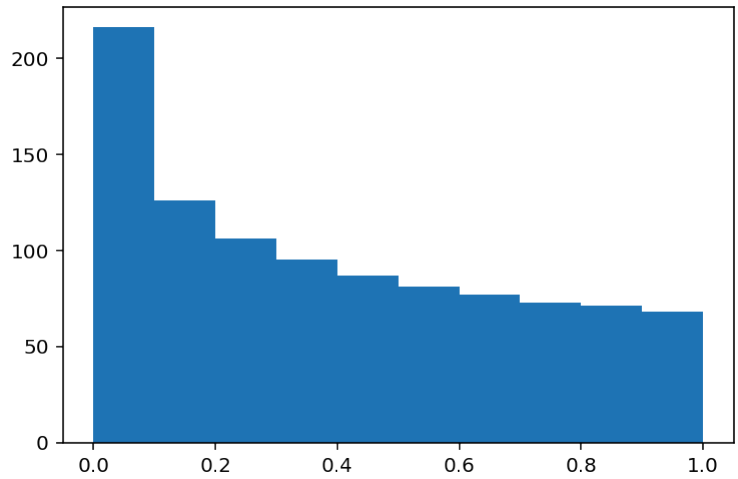
如下，加了分号之后只显示matplot绘制的图而不会返回其他数据

In [19]:
```
%matplotlib inline
from matplotlib import pyplot as plt
import numpy
x = numpy.linspace(0, 1, 1000)**1.5
plt.hist(x)
```

Out[19]:
```
(array([216., 126., 106.,  95.,  87.,  81.,  77.,  73.,  71.,  68.]),
 array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. ]),
 <a list of 10 Patch objects>)
```



In [20]:
```
plt.hist(x);
```



执行Shell命令

在命令前加`!`即可直接在Notebook中执行

```
# 列出所有ipynb文件
!ls *.ipynb
#安装plotly
!pip install plotly
```

使用Latex公式

In [21]:
```
r"\\( P(A \mid B) = \frac{P(B \mid A) \, P(A)}{P(B)} \\)"
```

Out[21]:
```
'\\\\( P(A \\mid B) = \\frac{P(B \\mid A) \\, P(A)}{P(B)} \\\\)'
```

$P(A \mid B) = \frac{P(B|A)\,P(A)}{P(B)}$

行间公式

```
$$P(A \mid B) = \frac{P(B \mid A) \, P(A)}{P(B)} $$
```

$$P(A \mid B) = \frac{P(B \mid A) \, P(A)}{P(B)}$$

行内公式

```
$P(A \mid B) = \frac{P(B \mid A) \, P(A)}{P(B)} $
```

$P(A \mid B) = \frac{P(B|A)\,P(A)}{P(B)}$

Notebook内用不同kernel运行代码Run code from a different kernel in a notebook

选项

- %%bash
- %%HTML
- %%python2
- %%python3
- %%ruby
- %%perl

安装其他kernel

比如安装R

```
conda install -c r r-essentials
```

同时运行R和Python代码

使用`rpy2`模块

用其他语言写部分函数Writing functions in other languages

安装 cython 和 fortran-magic

```
!pip install cython fortran-magic
```

In [22]:
```
%load_ext Cython
```

In [23]:
```
%%cython

def myltiply_by_2(float x):
    return 2.0 * x
```

In [24]:
```
myltiply_by_2(23.)
```

Out[24]:
```
46.0
```

多光标

按住`Ctrl`鼠标左键单击即可

Jupyter 插件

安装并激活插件和插件管理器

```
!pip install jupyter_contrib_nbextensions
!pip install jupyter_nbextensions_configurator
!jupyter contrib nbextension install --user
!jupyter nbextensions_configurator enable --user
```

利用Jupyter Notebook制作Presentation

安装RISE插件并激活

```
pip install RISE
jupyter-nbextension install rise --py --sys-prefix
jupyter-nbextension enable rise --py --sys-prefix
```

Jupyter输出系统The Jupyter output system

Jupyter Notebook 的输出系统相当强大

例如显示文件夹内的图片

In [25]:

```
import os
from IPython.display import display, Image
names = [f for f in os.listdir('./img/') if f.endswith('.png') or f.endswith('.jp
g')]
for name in names:
    display(Image('./img/' + name, width=100))
```

大数据处理'Big data' analysis

应对大数据场景的一些方案

- ipyparallel (ipython cluster)
- pyspark
- spark-sql magic `%%sql`

共享笔记 Sharing notebooks

- 通过菜单`File > Download as > HTML`将笔记转换为Html文件
- 通过菜单`File > Download as > PDF`将笔记转换为PDF文件
- 通过Gist 或将文件上传到GitHub
- jupyterhub
- 通过nbviewer在线渲染GitHub或DropBox上的文件
- 利用Pelican和ipynb文件搭建个人博客 (https://www.dataquest.io/blog/how-to-setup-a-data-science-blog/)

补充

自动补全

使用`Tab`键

更多信息以及文中介绍的一些资源网站链接，参考原文 (https://www.dataquest.io/blog/jupyter-notebook-tips-tricks-shortcuts/)

- « 魔方 (//zodiac911.github.io/blog/magic-cube.html)
- Jupyter Notebook内的Plotly绘图 (//zodiac911.github.io/blog/plotly-plot.html) »

Published

10 10, 2018

Category

posts (//zodiac911.github.io/blog/categories.html#posts-ref)

Tags

- ipynb 1 (//zodiac911.github.io/blog/tags#ipynb-ref)
- Jupyter Notebook 4 (//zodiac911.github.io/blog/tags#jupyter-notebook-ref)
- Python 16 (//zodiac911.github.io/blog/tags#python-ref)

Contact

(i) (t)

- Zodiac Wang - A Fantastic Learner
- Powered by Pelican (http://getpelican.com/). Theme: Elegant (https://github.com/Pelican-Elegant/elegant/)